Figure B: Avalanche Validator Lifecycle Model v0.8.0 — Detailed Lifecycle Execution

Internal Execution Architecture of PSUB 0 (p_lifecycle): Stage Coupling, Intermediate Outputs, Common Policy Signal, and Parallel State Updates

Pre-PSUB State & Day Context Initialization (evaluated on pre-transition state X_t)

Active Commitments `C_t`

Out-of-Set Records `O_t`

Global Supply `Z_t = (S_t)`

Day Context & Yield Signal Evaluation: G(S_t ; D_menu, ψ_prot)

Constructs `Day(t, start, draw, blocks, renewal_gap, entry_gap, settings)`. Evaluates yield feedback:

- `Renewal Gap: g_t^(R) = offered_rate(d_tenure; S_t, ψ_prot) - D_menu_renewal`
- `Entry Gap: g_t^(E) = offered_rate(14d; S_t, ψ_prot) - D_menu_entry`

Scheduled Queue
`Q_t (exogenous)`
Injected at Stage 6

Protocol Bounds & Rule
`ψ_prot (S_min=2k, S_max=3M)`
ACP-285 minting formula

Historical Benchmark
`D = (D_menu, tables)`
Reference rates & quantiles

Bootstrap Parameters
`θ_r & PRNG streams ξ_(r,t)`
Fitted per run & day

`Q_t enters at Assembly only`

Sequential Stage Execution in run_stages(stages, state, day)

Each stage receives (state, day, earlier); earlier dictionary stores preceding stage outputs. Executed sequentially inside p_lifecycle.

`reads C_t (ending commitments)`

`reads O_t (eligible out-of-set records)`

`g_t^(R) feeds Renewal`

`g_t^(E) feeds Arrivals`

1. Renewal Stage

`renewal_stage`

P(R=1) = σ(x^T β + ρ_R g_t^(R) + δ_r)

Reads: ending in C_t, g_t^(R) (node_record=0)

`Emits: ending, renews, ended`

`earlier["renewal"]`

2. Returns Stage

`returns_stage`

h_(j,t) (6 day bins, 8..365d, stake, history)

Reads: O_t (eligible: τ >= 8d), no yield response

`Emits: out_days, comes_back`

`earlier["returns"]`

3. Arrivals Stage

`arrivals_stage`

A_t ~ NegBin(μ_t ≤ 100.0, φ)

Reads: g_t^(E) entry yield gap

`Emits: count (A_t), node_key`

Participation Candidates N_t = (renews, comes_back, arrivals) pool sequentially into Continuation

4. Continuation Stage (_stake_continuation)

Executes both duration selection & stake choice within the single stage

Part A: Duration Selection d' ∈ [14, 365]
Empirical transition (4 previous bins) for renewals/returns; first_days for arrivals

Part B: Stake Choice s' ∈ [2k, 3M] AVAX (StakeChoice)
Hurdle p_(c,b,q) + 10-band log-ratio quantiles (50-observation fallback threshold)

`earlier["continuation"]`

5. Rewards Stage

`rewards_stage (Deterministic)`

I_(m,t) = potential_reward(s', d', S_t, period)

Reads: started commitments from Continuation, S_t

`Emits: issued_navax, started_stake_navax, days, rate`

`earlier["rewards"]`

6. Assembly Stage

`assembly_stage (Deterministic)`

Mutates C_(t+1), O_(t+1); injects Q_t

Reads: state, renewal, returns, started, sched Q_t

`Emits: Emits complete terminal policy signal dictionary`

COMMON POLICY SIGNAL = terminal output of assembly_stage

State-Update Function Bank: Exactly 24 Functions Evaluated in Parallel on Pre-PSUB State X_t

cadCAD execution semantics: each update function receives (params, substep, history, state=X_t, signal). Parallel evaluation, not sequential mutation.

Group 1: Active Commitment Micro-State (C_(t+1)) [8 Functions]

`Keys: stakes, days_left, done, after_return, days, handed_on, returns, node_key`

Implementation Logic: s_copy(name) updates each slice from signal. Sets days_left=days for new starts, decrements continuing.

Group 2: Out-of-Set Population Micro-State (O_(t+1)) [6 Functions]

`Keys: out_stakes, out_days, out_done, out_last_days, out_returns, out_node_key`

Implementation Logic: s_copy(name) updates out-of-set records. Increments out_days for retained nodes; appends new unrenewed exits.

Group 3: Persistent Global Protocol State (Z_(t+1)) [1 Functions]

`Keys: supply_navax`

Implementation Logic: s_supply evaluates state["supply_navax"] + signal["issued_navax"]. Strictly reads pre-PSUB supply S_t.

Group 4: State Readout Metrics & Concentration [4 Functions]

`Keys: active_validators, active_validator_stake_navax, returning_nodes, nakamoto_third_nodes`

Implementation Logic: s_count (len(stakes)), s_stake (sum(stakes)), s_out (len(out_stakes)), s_nakamoto (decentralization over node_keys).

Group 5: Intermediate Transition Flows & Diagnostics (Φ_t) [5 Functions]

`Keys: issued_navax, yield_gap_pp, started_stake_navax, started_stake_days_navax, offered_rate_30d`

Implementation Logic: s_copy(name) passes daily transition flux quantities into simulation record for observable metric export.

Reconciled Function Count: Exactly 24 Functions

`8 (Commitment) + 6 (Out-of-Set) + 1 (Supply) + 4 (Readouts) + 5 (Flows) = 24`

Visual Grammar & Implementation Traceability:

- Diamonds (Muted Pink): Stochastic behavioural transition policies. • Pentagons (Sage Green): Deterministic protocol mechanisms & state-update operations (*adapted convention).
- Stage Order: Stages 1–6 execute sequentially inside p_lifecycle; earlier outputs feed subsequent stages. All 24 state updates evaluate in parallel against pre-PSUB state X_t.